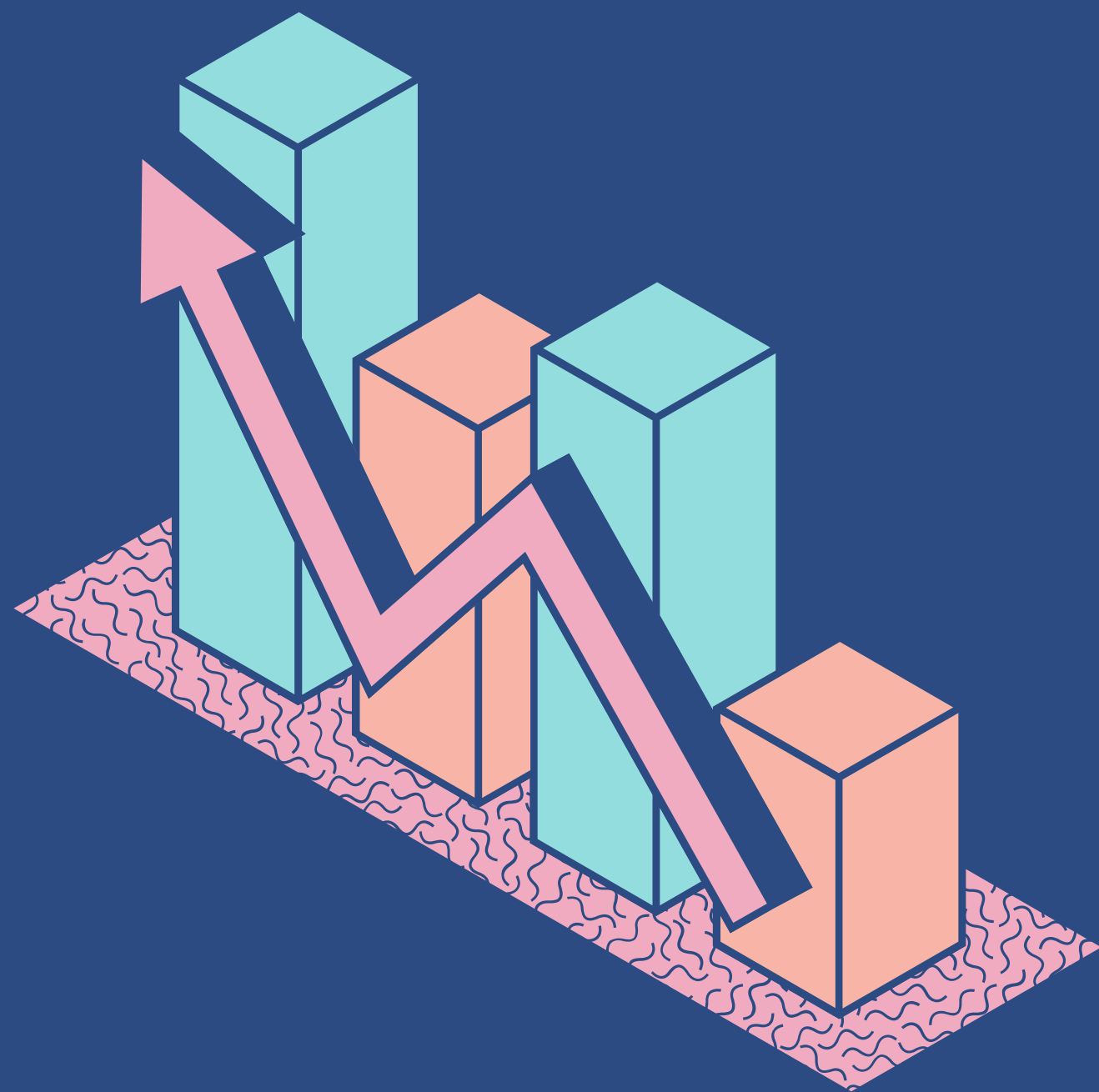




Forecasting Model Development and Optimization

Walmart Sales Forecast overview

Problem:

There are many seasons that sales are significantly higher or lower than averages. If the company does not know about these seasons, it can lose too much money. Predicting future sales is one of the most crucial plans for a company. Sales forecasting gives an idea to the company for arranging stocks, calculating revenue, and deciding to make a new investment. Another advantage of knowing future sales is that achieving predetermined targets from the beginning of the seasons can have a positive effect on stock prices and investors' perceptions. Also, not reaching the projected target could significantly damage stock prices, conversely. And, it will be a big problem especially for Walmart as a big company.

Aim:

My aim in this project is to build a model which predicts sales of the stores. With this model, Walmart authorities can decide their future plans which is very important for arranging stocks, calculating revenue and deciding to make new investment or not.

Solution:

**With the accurate prediction company can;
Determine seasonal demands and take action for this**

**Protect from money loss because achieving sales targets can have a positive
effect on stock prices and investors' perceptions**

Forecast revenue easily and accurately

Manage inventories

Do more effective campaigns

Plan:

Data collection

From Kaggle :

[https://www.kaggle.com/datasetsaslanahmedov/walmart-sales-forecast](https://www.kaggle.com/datasets/aslanahmedov/walmart-sales-forecast)

Data Dictionary

Store

Date

Temperature

Fuel_Price

MarkDown1–5

CPI

Unemployment

IsHoliday

Load Data

```
df = pd.read_csv('features.csv')
```

Describe Data

```
df.describe()
```

Distribution Analysis

OUTLIER DETECTION

HANDLE

NAN

VALUES

CORRELATION ANALYSIS

TIME SERIES ANALYSIS

DATA VISIUALIZATION

Loaded training, testing, and features datasets from CSV files.
Merged datasets on common keys: Store, Date, and IsHoliday.

```
# Load datasets and Merge datasets
train=pd.read_csv('/content/train.csv')
features=pd.read_csv('/content/features.csv')
test=pd.read_csv('/content/test.csv')
train_data= train.merge(features, on=['Store', 'Date', 'IsHoliday'], how='left')
test_data= test.merge(features, on=['Store', 'Date', 'IsHoliday'], how='left')
```

Convert raw date strings to datetime objects

Extract meaningful features from dates

Prepare data for machine learning models

```
# Convert date columns to datetime
for df in [train, test, features]:
    df['Date'] = pd.to_datetime(df['Date'])
```

```
# Extract date-related features
for df in [train_data, test_data]:
    df['Date'] = pd.to_datetime(df['Date'])
    df['Year'] = df['Date'].dt.year
    df['Month'] = df['Date'].dt.month
    df['WeekOfYear'] = df['Date'].dt.isocalendar().week
```

```
# Drop original Date
train_data.drop(['Date'], axis=1, inplace=True)
test_data.drop(['Date'], axis=1, inplace=True)
```

Prepare data for machine learning models

Prevent errors in analysis and modeling

```
# Fill missing markdown values with 0
markdown_cols = ['MarkDown1', 'MarkDown2', 'MarkDown3', 'MarkDown4', 'MarkDown5']
train_data[markdown_cols] = train_data[markdown_cols].fillna(0)
test_data[markdown_cols] = test_data[markdown_cols].fillna(0)

# Fill missing CPI and Unemployment values using forward fill
train_data[['CPI', 'Unemployment']] = train_data[['CPI', 'Unemployment']].ffill()
test_data[['CPI', 'Unemployment']] = test_data[['CPI', 'Unemployment']].ffill()
```


Separate features (X) from target variable (y)

Prepare test dataset for predictions

Ensure clean, structured data

```
# Define X and y for training
X = train_data.drop(['Weekly_Sales'], axis=1)
y = train_data['Weekly_Sales']

# Define test dataset for prediction
z_test = test_data.copy()
```

Evaluate model performance
Prevent overfitting
Simulate real-world prediction scenarios

```
# تقسيم البيانات  
x_train, x_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

The Importance of Normalization

1. Brings features to same scale
2. Prevents dominance of large-value features
3. Improves model performance
4. Essential for many machine learning algorithms

```
scaler = StandardScaler()  
x_train_scaled = scaler.fit_transform(x_train)  
x_test_scaled = scaler.transform(x_test)  
z_test_scaled = scaler.transform(z_test)
```

Random Forest Regressor model

What is Random Forest Regressor?

1. Ensemble learning method
2. Used for regression problems
3. Combines multiple decision trees
4. Provides robust and accurate predictions

Why Choose Random Forest?

1. Handles non-linear relationships
2. Reduces overfitting
3. Works well with large datasets
4. Provides feature importance
5. Handles missing values
6. Robust to outliers



```
# Train the RandomForestRegressor model
param_grid = {
    'n_estimators': [100, 200],
    'max_depth': [10, 20, None],
    'min_samples_split': [2, 5],
    'min_samples_leaf': [1, 2],
}

rf = RandomForestRegressor(random_state=42)

grid_search = GridSearchCV(
    estimator=rf,
    param_grid=param_grid,
    cv=3,
    n_jobs=-1,
    verbose=2
)

▶ grid_search.fit(x_train_scaled, y_train)
```


Why Evaluate Machine Learning Models?

1. Assess predictive performance
2. Understand model strengths and limitations
3. Guide model improvement
4. Ensure reliable predictions

Regression Metrics:

1. Mean Squared Error (MSE)

Measures average squared difference

Penalizes larger errors more heavily

2. R-squared (R^2) Score

Indicates proportion of variance explained

Range: 0 to 1 Higher is better

```
y_pred = grid_search.predict(x_test_scaled)
```

```
mse = mean_squared_error(y_test, y_pred)
```

```
r2 = r2_score(y_test, y_pred)
```

```
print("tuned hpyerparameters :(best parameters) ",grid_search.best_params_)
```

```
print("best_score_RF :",grid_search.best_score_)
```

```
print("mse_RF :",mse)
```

```
print("r2_RF :",r2)
```

Support Vector Regression (SVR)

What is SVR?

1. Extension of Support Vector Machines (SVM)
2. Powerful regression technique
3. Works well with non-linear relationships
4. Effective in high-dimensional spaces

Why Choose SVR?

1. Handles non-linear relationships
2. Works well with high-dimensional data
3. Less sensitive to outliers
4. Flexibility through kernel selection
5. Robust in complex prediction scenarios


```
#create Support vector regression
param_grid = {
    'kernel': ['rbf', 'poly'],
    'C': [0.1, 1, 10],
    'epsilon': [0.01, 0.1, 0.2],
    'gamma': ['scale', 'auto'] }

|
#GridSearchCV
svr = SVR()

grid_search = GridSearchCV(
    estimator=svr,
    param_grid=param_grid,
    cv=3,
    scoring='neg_root_mean_squared_error',
    n_jobs=-1,
    verbose=2
)
grid_search.fit(x_train_scaled, y_train)
```

Predict Model

```
y_pred = grid_search.predict(x_test_scaled)
```

```
mse = mean_squared_error(y_test, yhat)  
r2 = r2_score(y_test, yhat)  
print("tuned hpyerparameters :(best parameters) ",svr_cv.best_params_)  
print("best_score_RF :",grid_search.best_score_)
```

Making Predictions & Creating a Comprehensive Results DataFrame

```
test_preds = model.predict(X_test)

# Preview predictions
print(test_preds[:10])

# Prepare results in DataFrame format
predictions_table = pd.DataFrame({
    'Store': test_data['Store'],
    'Dept': test_data['Dept'],
    'Year': test_data['Year'],
    'Month': test_data['Month'],
    'WeekOfYear': test_data['WeekOfYear'],
    'IsHoliday': test_data['IsHoliday'],
    'Predicted_Weekly_Sales': test_preds
})

# Display the first 10 rows as a table
predictions_table.head(10)
```

Hugging Face is way more fun with friends and colleagues! [Join an organization](#)

Dismiss this message

Sales Forecasting & Optimization

Upload Walmart sales data and predict future sales by store and department.

Upload Sales CSV File

train.csv12.2 MB

Store Number

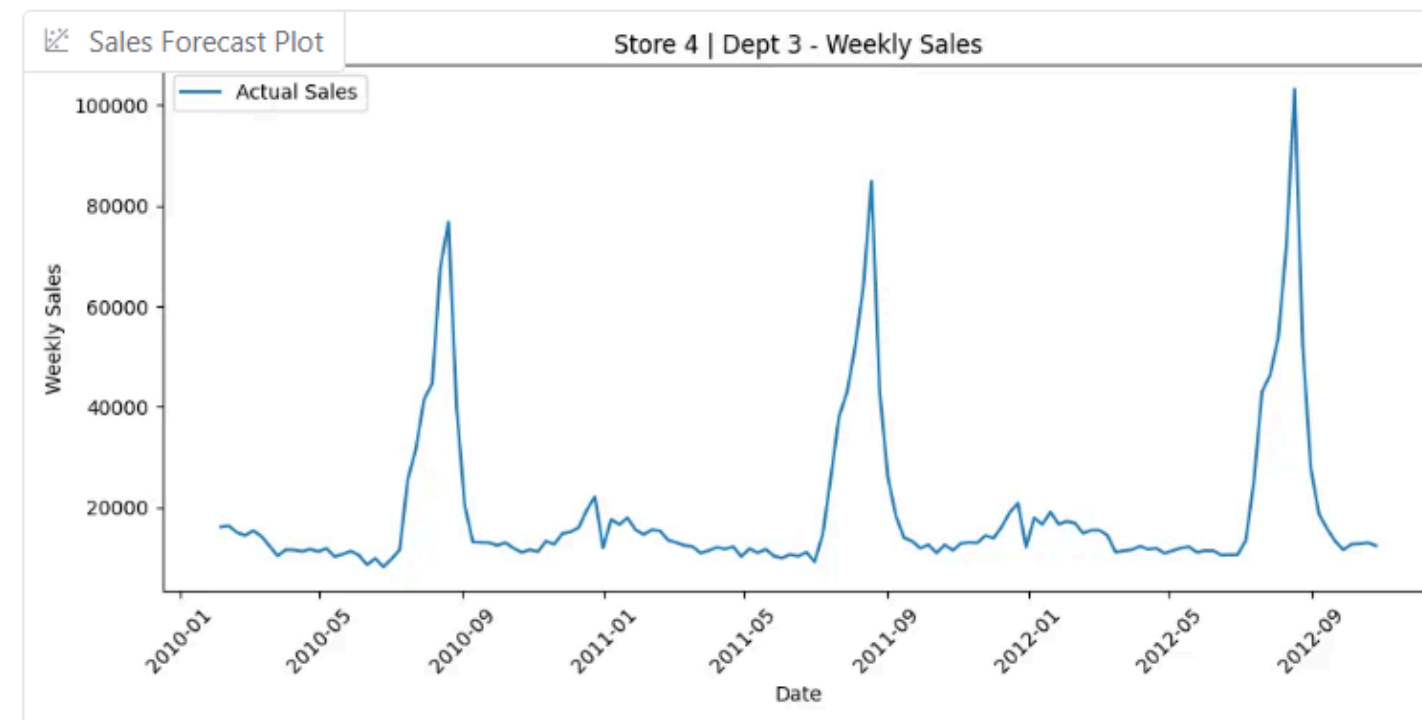
4

Department Number

3

Clear

Submit



Error (MAE)

MAE: 3976.35

Share via Link

https://habiba12-sales.hf.space/?logs=container&__theme=system&deep_link=yD1Hvw6werw